
MLP Coursework 1

S2026994

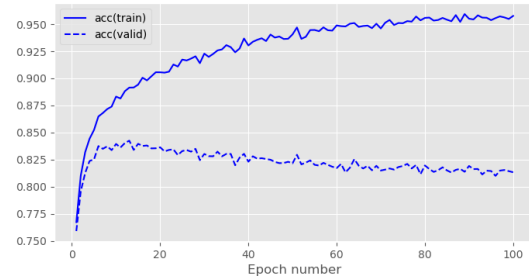
Abstract

In this report we study the problem of overfitting, which is the training regime where performance increases on the training set but decrease on validation data. Overfitting prevents our trained model from generalizing well to unseen data. We first analyse the given example and discuss the probable causes of the underlying problem. Then we investigate how the depth and width of a neural network can affect overfitting in a feedforward architecture and observe that increasing width and depth tend to enable further overfitting. Next we discuss how two standard methods, Dropout and Weight Penalty, can mitigate overfitting, then describe their implementation and use them in our experiments to reduce the overfitting on the EMNIST dataset. Based on our results, we ultimately find that both dropout and weight penalty are able to mitigate overfitting. Finally, we conclude the report with our observations and related work. Our main findings indicate that preventing overfitting is achievable through regularization, although combining different methods together is not straightforward.

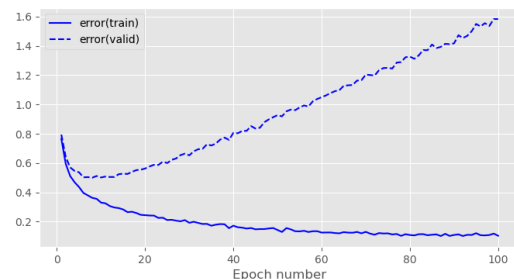
1. Introduction

In this report we focus on a common and important problem while training machine learning models known as overfitting, or overtraining, which is the training regime where performances increase on the training set but decrease on unseen data. We first start with analysing the given problem in Figure 1, study it in different architectures and then investigate different strategies to mitigate the problem. In particular, Section 2 identifies and discusses the given problem, and investigates the effect of network width and depth in terms of generalization gap (see Ch. 5 in Goodfellow et al. 2016) and generalization performance. Section 3 introduces two regularization techniques to alleviate overfitting: Dropout (Srivastava et al., 2014) and L1/L2 Weight Penalties (see Section 7.1 in Goodfellow et al. 2016). We first explain them in detail and discuss why they are used for alleviating overfitting. In Section 4, we incorporate each of them and their various combinations to a three hidden layer¹ neural network, train it on the EMNIST dataset, which contains 131,600 images of characters and digits,

¹We denote all layers as hidden except the final (output) one. This means that depth of a network is equal to the number of its hidden layers + 1.



(a) accuracy by epoch



(b) error by epoch

Figure 1. Training and validation curves in terms of classification accuracy (a) and cross-entropy error (b) on the EMNIST dataset for the baseline model.

each of size 28x28, from 47 classes. We evaluate them in terms of generalization gap and performance, and discuss the results and effectiveness of the tested regularization strategies. Our results show that both dropout and weight penalty are able to mitigate overfitting.

Finally, we conclude our study in Section 5, noting that preventing overfitting is achievable through regularization, although combining different methods together is not straightforward.

2. Problem identification

Overfitting to training data is a very common and important issue that needs to be dealt with when training neural networks or other machine learning models in general (see Ch. 5 in Goodfellow et al. 2016). A model is said to be overfitting when as the training progresses, its performance on the training data keeps improving, while its is degrading on validation data. Effectively, the model stops learning related patterns for the task and instead starts to memorize specificities of each training sample that are irrelevant to new samples. Overfitting leads to bad generalization per-

# Hidden Units	Val. Acc.	Train Error	Val. Error
32	78.8	0.525	0.705
64	80.5	0.336	0.678
128	80.8	0.161	0.936

Table 1. Validation accuracy (%) and training/validation error (in terms of cross-entropy error) for varying network widths on the EMNIST dataset.

formance in unseen data, as performance on validation data is indicative of performance on test data and (to an extent) during deployment.

Although it eventually happens to all gradient-based training, it is most often caused by models that are too large with respect to the amount and diversity of training data. The more free parameters the model has, the easier it will be to memorize complex data patterns that only apply to a restricted amount of samples. A prominent symptom of overfitting is the generalization gap, defined as the difference between the validation and training error. A steady increase in this quantity is usually interpreted as the model entering the overfitting regime.

Figure 1a and 1b show a prototypical example of overfitting. We see in Figure 1a that [the train and validation accuracies initially increase as a function of epoch number. The same behaviour can be seen in figure 1b were train and validation errors are decreasing with one another. This observation is indicative of a model that is generalising well, as described in (Goodfellow et al., 2016, Ch. 5.2). At approximately 10 epochs one can see a sharp turn in the validation error. From this point on the errors diverge. This is typical of overparamaterised networks and is indicative of over-fitting (Goodfellow et al., 2016, Ch. 5.2). From this point on the train validation error gap increases with epoch number. Even when a network is generalising well, one still expects to see a small gap between the train and validation error. The fact that the validation error is increasing does however indicate that the model is overfitting.] .

The extent to which our model overfits depends on many factors. For example, the quality and quantity of the training set and the complexity of the model. If we have sufficiently many diverse training samples, or if our model contains few hidden units, it will in general be less prone to overfitting. Any form of regularization will also limit the extent to which the model overfits.

2.1. Network width

[Question Table 1 - Fill in Table 1 with the results from your experiments varying the number of hidden units.] [Question Figure 2 - Replace the images in Figure 2 with figures depicting the accuracy and error, training and validation curves for your experiments varying the number of hidden units.]

First we investigate the effect of increasing the number of

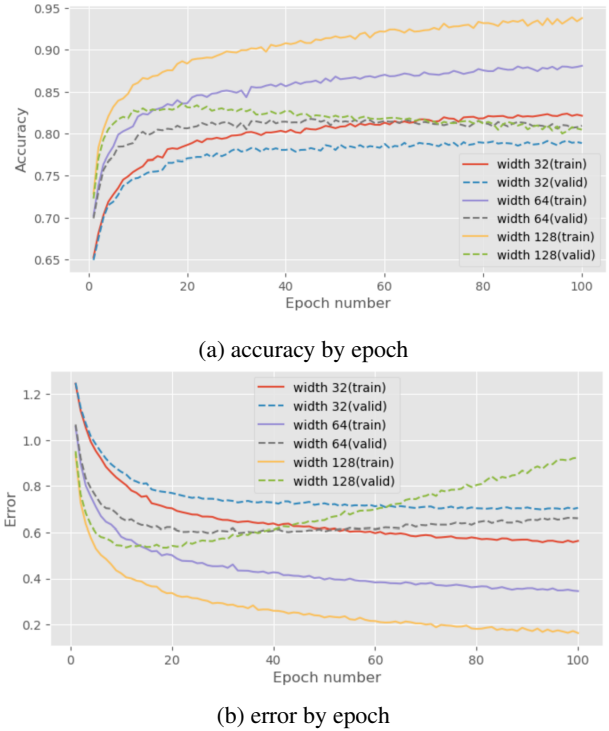


Figure 2. Training and validation curves in terms of classification accuracy (a) and cross-entropy error (b) on the EMNIST dataset for different network widths.

hidden units in a single hidden layer network when training on the EMNIST dataset. The network is trained using the Adam optimizer with a learning rate of 9×10^{-4} and a batch size of 100, for a total of 100 epochs.

The input layer is of size 784, and output layer consists of 47 units. Three different models were trained, with a single hidden layer of 32, 64 and 128 ReLU hidden units respectively. Figure 2 depicts the error and accuracy curves over 100 epochs for the model with varying number of hidden units. Table 1 reports the final accuracy, training error, and validation error. We observe that [the network with a hidden layer width of 32 units is the only network were overfitting is not apparent. This is indicated by the fact that both the train and validation errors are decreasing as a function of epoch number (Goodfellow et al., 2016, Ch. 5.2), despite a gap still being present. This is also apparent in the accuracy curves, were both the train and validation accuracies initially increase sharply before flattening off to a small positive gradient. Both the 64 and 128 width networks show this trend in their training curves. Their validation curves however show signs of overfitting due to their increasing value and divergence from the train error curve. One can see similarities between the overfit model in figure 1 and these models, namely that the validation errors deflect from a minima early on in the training and increase with subsequent epochs. The 128 width network shows this phenomena to a much greater degree, with a train validation error gap that is far more prominent than the 64 width network, as shown in table 1.] .

[From the data observed in figure 2 and table 1, it does seem that network width has an effect on the degree to which the model overfits to the training data. Namely, that the wider networks overfit to a greater degree. This is in agreement with the general literature of the subject (as discussed in (Bejani & Ghatee, 2021)). Wider, over-paramaterised neural networks have a greater capacity to explicitly remember the training data, even up to the point of remembering single, randomly mislabelled datapoints (Zhang et al., 2016). Hence they can over-fit by simply memorising all datapoints.

From observations made in this section, it is however not possible to determine if it's the width alone that causes this phenomena, or the fact that increasing the width introduces more free parameters that in turn causes the effect. It has generally been found that it's the number of model parameters (or more specifically model capacity) that increases the networks propensity for over-fitting (Goodfellow et al., 2016, Ch. 5.2, P. 112 - 113).

Interestingly, it is found that in the infite width limit neural networks have vastly different learning dynamics than in the finite limit (Lee et al., 2019). These models can approximate to guassian process and can in fact lead to better generalisation than models in the large width over-fitting regime. This has been verified empirically for very wide models. For our case, it is most likely that our models in the region of parameter space were they are over fitting, but not wide enough to operate in the regime described above.

The final observation to note is that the validation error increases by approximately 2 to 3 times its optimal generalization value (located at approximately 10 ephocs), whereas the the validation accuracy barely decreases from its largest value. Here, the overfitting is manifesting as a probabilistic error (as discussed in (Guo et al., 2017)). The model is generally still predicting the correct label, however due to the use of the cross entropy error, the models confidence in this prediction factors greatly into the error term, not just its value. The over-fit model is assigning larger probabilities to incorrect labels and lower probabilities to the correct labels than the well generalised model is. This is why the validation error of the 64 and 128 width models are far larger that of the 32 width model but the accuracies are not significantly smaller.

].

2.2. Network depth

[Question Table 2 - Fill in Table 2 with the results from your experiments varying the number of hidden layers.] [Question Figure 3 - Replace these images with figures depicting the accuracy and error; training and validation curves for your experiments varying the number of hidden layers.]

Next we investigate the effect of varying the number of

# Hidden Layers	Val. Acc.	Train Error	Val. Error
1	80.4	0.167	0.942
2	81.9	0.092	1.500
3	81.9	0.141	1.660

Table 2. Validation accuracy (%) and training/validation error (in terms of cross-entropy error) for varying network depths on the EMNIST dataset.

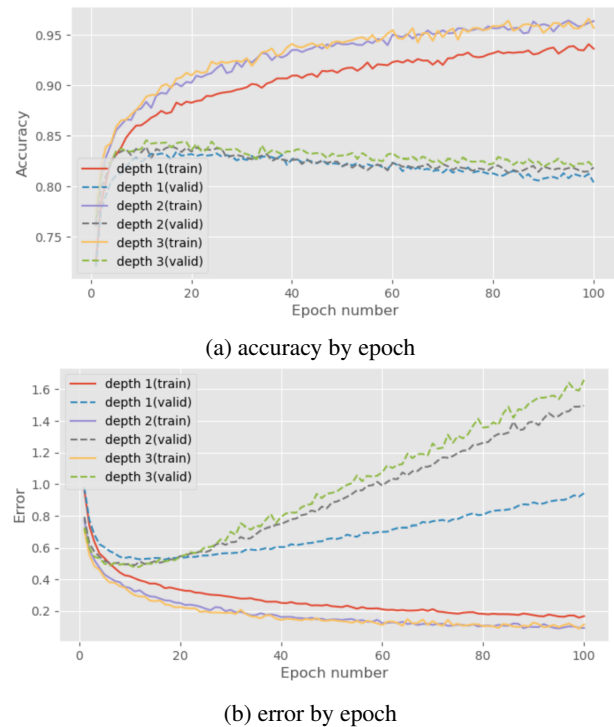


Figure 3. Training and validation curves in terms of classification accuracy (a) and cross-entropy error (b) on the EMNIST dataset for different network depths.

hidden layers in the network. Table 2 and Figure 3 depict results from training three models with one, two and three hidden layers respectively, each with 128 ReLU hidden units. As with previous experiments, they are trained with the Adam optimizer with a learning rate of 9×10^{-4} and a batch size of 100.

We observe that [the validation error curves in figure 3b still show an increasing divergence from the train error curves for all 3 models. This is evidence that the models are still operating within the overfitting regime for the 3 depths investigated. It's interesting to note that the gradient of the depth 2 model's validation error is much larger than that of the depth 1 model, but the difference between the depth 2 and depth 3 models is very small. This is also indicated by the similar train validation error gaps in table 2.]

[The findings above, that increasing the network depth increases the severity of over-fitting is consistent with the subject literature (Goodfellow et al., 2016, Ch. 5.2). The effect of increasing network depth does however

not seem to increase the severity of over fitting as much as increasing width.

One explanation for this may simply stem from the fact that the depth 2 model may be at its maximum extent of over-paramaterisation. That is, the model has sufficient capacity to perfectly remember the training dataset (such as the models used in (Zhang et al., 2016)). Adding more parameters to such a model increases it's capacity, but since the data is already memorised it's not going to substantially change the predictions of the trained model. For example, a quadritic model fitting 2 points, and a cubic model fitting 2 points could end up being the same, as the cubic model may train to set it's cubic exponent to 0.

There is empirical evidence that very deep networks of the a similar complexity to a more traditional shallower models do tend to act better as universal approximators (He et al., 2015), so one may expect the deeper networks to over-fit the data to a greater degree. The fact that the depth 2 network is already very deep in the over fitting regieme means that this effect will not most likely increase as much in severity for subsequent depths.

It would ultimately be interesting to see what effect the depth and width have when the number of network paramaters are kept the same. It may be the case that a for a very narrow network bottlenecking (Matejka et al., 2014)sets in and the network can again generalise better.] .

3. Regularization

In this section, we investigate three regularization methods to alleviate the overfitting problem, specifically dropout layers, the L1 and L2 weight penalties and label smoothing.

3.1. Dropout

Dropout (Srivastava et al., 2014) is a stochastic method that randomly inactivates neurons in a neural network according to an hyperparameter, the inclusion rate (*i.e.* the rate that an unit is included). Dropout is commonly represented by an additional layer inserted between the linear layer and activation function. Its forward pass during training is defined as follows:

$$\text{mask} \sim \text{bernoulli}(p) \quad (1)$$

$$\mathbf{y}' = \text{mask} \odot \mathbf{y} \quad (2)$$

where $\mathbf{y}, \mathbf{y}' \in \mathbb{R}^d$ are the output of the linear layer before and after applying dropout, respectively. $\text{mask} \in \mathbb{R}^d$ is a mask vector randomly sampled from the Bernoulli distribution with inclusion probability p , and $\odot$ denotes the element-wise multiplication.

At inference time, stochasticity is not desired, so no neurons are dropped. To account for the change in expectations of the output values, we scale them down by the inclusion

probability p :

$$\mathbf{y}' = \mathbf{y} * p \quad (3)$$

As there is no nonlinear calculation involved, the backward propagation is just the element-wise product of the gradients with respect to the layer outputs and mask created in the forward calculation. The backward propagation for dropout is therefore formulated as follows:

$$\frac{\partial \mathbf{y}'}{\partial \mathbf{y}} = \text{mask} \quad (4)$$

Dropout is an easy to implement and highly scalable method. It can be implemented as a layer-based calculation unit, and be placed on any layer of the neural network at will. Dropout can reduce the dependence of hidden units between layers so that the neurons of the next layer will not rely on only few features from of the previous layer. Instead, it forces the network to extract diverse features and evenly distribute information among all features. By randomly dropping some neurons in training, dropout makes use of a subset of the whole architecture, so it can also be viewed as bagging different sub networks and averaging their outputs.

3.2. Weight penalty

L1 and L2 regularization (Ng, 2004) are simple but effective methods to mitigate overfitting to training data. The application of L1 and L2 regularization strategies could be formulated as adding penalty terms with L1 and L2 norm square of weights in the cost function without changing other formulations. The idea behind this regularization method is to penalize the weights by adding a term to the cost function, and explicitly constrain the magnitude of the weights with either the L1 and L2 norms. The optimization problem takes a different form:

$$\text{L1: } \min_{\mathbf{w}} E_{\text{data}}(\mathbf{X}, \mathbf{y}, \mathbf{w}) + \lambda \|\mathbf{w}\|_1 \quad (5)$$

$$\text{L2: } \min_{\mathbf{w}} E_{\text{data}}(\mathbf{X}, \mathbf{y}, \mathbf{w}) + \lambda \|\mathbf{w}\|_2^2 \quad (6)$$

where E_{data} denotes the cross entropy error function, and $\{\mathbf{X}, \mathbf{y}\}$ denotes the input and target training pairs. λ controls the strength of regularization.

Weight penalty works by constraining the scale of parameters and preventing them to grow too much, avoiding overly sensitive behaviour on unseen data. While L1 and L2 regularization are similar to each other in calculation, they have different effects. Gradient magnitude in L1 regularization does not depend on the weight value and tends to bring small weights to 0, which can be used as a form of feature selection, whereas L2 regularization tends to shrink the weights to a smaller scale uniformly.

3.3. Label smoothing

Label smoothing regularizes a model based on a softmax with K output values by replacing the hard target 0 labels

and 1 labels with $\frac{\alpha}{K-1}$ and $1 - \alpha$ respectively. α is typically set to a small number such as 0.1.

$$\begin{cases} \frac{\alpha}{K-1}, & \text{if } t_k = 0 \\ 1 - \alpha, & \text{if } t_k = 1 \end{cases} \quad (7)$$

The standard cross-entropy error is typically used with these *soft* targets to train the neural network. Hence, implementing label smoothing requires only modifying the targets of training set. This strategy may prevent a neural network to obtain very large weights by discouraging very high output values.

4. Balanced EMNIST Experiments

[Question Table 3 - Fill in Table 3 with the results from your experiments for the missing hyperparameter values for each of L1 regularisation, L2 regularisation, Dropout and label smoothing (use the values shown on the table).]

[Question Figure 4 - Replace these images with figures depicting the Validation Accuracy and Generalisation Gap (difference between validation and training error) for each of the experiment results varying the Dropout inclusion rate, and L1/L2 weight penalty depicted in Table 3 (including any results you have filled in).]

Here we evaluate the effectiveness of the given regularization methods for reducing the overfitting on the EMNIST dataset. We build a baseline architecture with three hidden layers, each with 128 neurons, which suffers from overfitting as shown in section 2.

Here we train the network with a lower learning rate of 10^{-4} , as the previous runs were overfitting after only a handful of epochs. Results for the new baseline (c.f. Table 3) confirm that lower learning rate helps, so all further experiments are run using it.

Here, we apply the L1 or L2 regularization with dropout to our baseline and search for good hyperparameters on the validation set. Then, we apply the label smoothing with $\alpha = 0.1$ to our baseline. We summarize all the experimental results in Table 3. For each method except the label smoothing, we plot the relationship between generalization gap and validation accuracy in Figure 4.

First we analyze three methods separately, train each over a set of hyperparameters and compare their best performing results.

[Table 3 shows the result of the experiments ran using different regularisation techniques and hyperparameter values. In this discussion, I will denote the Dropout, L1 penalty, L2 Penalty hyperparameters as λ_d , λ_{L1} , and λ_{L2} respectively.

Dropout layers seem to help mitigate the effect of overfitting. One notes that the validation accuracy increases with dropout hyperparameter value. For $\lambda_d > 0.85$, the model beats the baseline for accuracy, and yields the joint best accuracy when $\lambda_d = 0.97$. Despite a vali-

dation train error gap still being present, it's substantially smaller than the baseline model, indicating that the model is generalising better. It is interesting to note that the gap increases as the model accuracy does. This relates to the explanation in section 2.1, the model is obtaining the highest probability for the correct label but is more uncertain about it. For EMNIST classification purposes, accuracy is the most important metric, so the dropout models past $\lambda_d = 0.85$ are considered to be better than the baseline.

The L1 regularization seems to universally be worse than the baseline model, both in terms of accuracy and validation error. For values of $\lambda_{L1} < 10^{-3}$ the accuracy is still reasonable, but the train and validation errors are both larger than the baseline values. For greater λ_{L1} values the validation accuracy drops to just above 2%. Noting that the dataset has 47 distinct classes, this is close to the accuracy that a random guess would yield. It's clear here that the model has failed to train at all. The most likely explanation is that L1 term in the gradient is causing the model to take steps in loss space that are too large, causing it overshoot minima when performing gradient descent. This is because the L1 gradient term is always constant, so the gradient descent step size is bounded from below by $|\lambda_d|$ for each parameter. It is however interesting to note that for all hyperparameter values, the train validation error gap has closed. For the 2 larger λ_{L1} values this is expected, as the model has failed to train. For the more accurate models this does indicate that the overfitting has been quelled, but unfortunately at the expense of model accuracy. It may be beneficial to investigate even smaller λ_{L1} values, as the increased train error values indicate that the L1 term is not allowing the model to descend deep enough into the loss space minima.

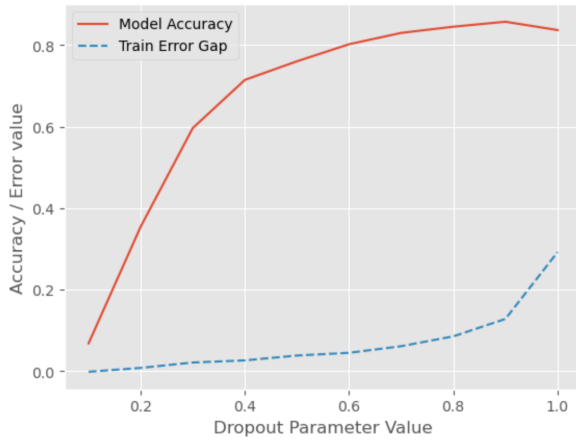
The introduction of an L2 regularizer into the model seems to cause better generalisation than the baseline model. This is indicated by a reduction in the train validation error gap and a slight increase in the model accuracy. For increasing λ_{L2} values, the gap does reduce, but at the expense of accuracy. This is again most likely due to regularizer not allowing the model to descend deep enough into the train loss space minima (as happened for the L1 model).

Finally, label smoothing seems to yield the joint best accuracy of all the models, but at the expense of both train and validation error. Recalling that the model outputs a probability mass function due to the final softmax term, the use of label smoothing can be interpreted as decreasing the train label confidence. This in turn causes the model to factor this uncertainty into its training, which as described earlier, manifests as an increased model error. It is clear however that this uncertainty has served to increase accuracy and generalization.

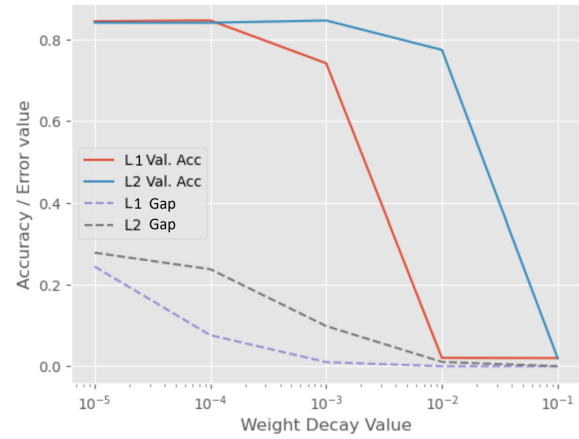
From the data gathered, models that beat the baseline accuracy are; the L2 model with ($\lambda_{L2} = 1e-3$ & $\lambda_{L2} =$

Model	Hyperparameter value(s)	Validation accuracy	Train Error	Validation Error
Baseline	-	0.837	0.241	0.533
Dropout	0.6	80.7	0.549	0.593
	0.7	82.8	0.447	0.508
	0.85	85.1	0.329	0.434
	0.97	85.4	0.244	0.457
L1 penalty	5e-4	79.5	0.642	0.658
	1e-3	74.0	0.871	0.879
	5e-3	2.41	3.850	3.850
	5e-2	2.20	3.850	3.850
L2 penalty	5e-4	85.1	0.306	0.460
	1e-3	85.1	0.357	0.451
	5e-3	81.3	0.586	0.607
	5e-2	39.2	2.258	2.256
Label smoothing	0.1	85.4	1.020	1.151

Table 3. Results of all hyperparameter search experiments. *italics* indicate the best results per series (Dropout, L1 Regularisation, L2 Regularisation, Label smoothing) and **bold** indicates the best overall.



(a) Accuracy and error by inclusion probability.



(b) Accuracy and error by weight penalty.

Figure 4. Accuracy and error by regularisation strength of each method (Dropout and L1/L2 Regularisation).

5e - 4), the label smoothing model with ($\alpha = 0.1$) and the dropout models for $\lambda_d = 0.97$ & $\lambda_d = 0.85$.

From this analysis, I have chosen to test the dropout model ($\lambda_d = 0.97$) on the test dataset. I consider this to be better than the Label Smoothing model purely due to the smaller validation error term. The dropout model is more confident in its prediction than the Label Smoothing model is whilst obtaining the same accuracy. A test accuracy and test error of 84.8% and 0.437 respectively was obtained. This is close to the validation values, and indicates that this model has indeed generalised well.

].

[For 8 subsequent model assessments the hyperparameter combinations I would use can be found in 4. For the below investigations, I would keep the learning rate the same. This is to allow one to directly compare the investigations with the baseline and the previously assessed

models.

- **Model 1:** I would propose this combination as these were the 2 best performing models. It would be interesting to see if combining the 2 models yields better accuracy, or if the 2 techniques oppose one another and cause worse generalization.
- **Model 2:** This is a combination of the best Dropout and best L2 model. I propose this investigation as it may be the case that some regularization techniques work better together than others.
- **Model 3:** The motivation for this investigation is the same as for the previous model. Its simply investigating another combination of high performing models.
- **Model 4:** This proposes testing the 3 best techniques together. If it is the case that the proposals

Model number	Model combination	Hyperparameter values
1	Dropout Label Smoothing	$\lambda_d = 0.97$ $\alpha = 0.1$
2	Dropout L2	$\lambda_d = 0.97$ $\lambda_{L2} = 5e - 4$
3	Label Smoothing L2	$\alpha = 0.1$ $\lambda_{L2} = 5e - 4$
4	Label Smoothing L2 Dropout	$\alpha = 0.1$ $\lambda_{L2} = 5e - 4$ $\lambda_d = 0.97$
5	Dropout	$\lambda_d = 0.985$
6	Dropout	$\lambda_d = 0.91$
7	Label Smoothing	$\alpha = 0.05$
8	Label Smoothing	$\alpha = 0.15$

Table 4. A table showing the hyperparameter and model combinations I would investigate for 8 subsequent runs.

above lead to better generalization, it may be the case that all 3 further increase the network performance.

- **Models 5& 6:** This proposed investigation serves to find the optimal value of λ_d . The models seem to get better for larger λ_d , although the baseline model is the same as $\lambda_d = 1$, so this cannot be the optimal value. This model is noticeably better ($\approx 2\%$), so I propose $\lambda_d = 0.985$ to scope the gap between this and the baseline model. I also propose a model with $\lambda_d = 0.91$ as this is in-between our existing datapoints. 0.97 is very close to 1, so it may well be the case that the optimal value of λ_d lies below 0.97.
- **Models 7 & 8:** I propose this investigation for the same reason. Label Smoothing seemed to be an effective technique to improve generalization and finding the optimal values of α would be beneficial. The reason I have proposed these specific values of α is to probe the space around the already evaluated model. As there is only one existing data point there isn't really a trend to follow.

General considerations for models 5,6,7 & 8: When performing such investigations, one needs to be careful they aren't fitting hyperparameters to the validation set. For this reason I would propose shuffling the training and validation sets between investigations, or possibly using K-Fold cross validation to attempt to mitigate this.

For further investigations I would propose testing other combinations to see if the optimal mixture and hyperparameter values can be found. It may be the case that suboptimal regularization techniques generalize better when employed with others, so testing these would be

insightful. It would also be interesting to further investigate the optimal values for the hyperparameters when the technique is employed on its own such as for models 5 - 8.

Finally, It may be interesting to see that happens if the baseline model is trained well beyond generalization. It was recently noticed that overparameterized models can suddenly generalize (and often very well) when trained far past the point of over fitting (Power et al., 2022). It could well be the case that this model exhibits the same behaviour, and if so, it would be good to assess how well it generalizes. This is obviously a very inefficient way of training a model, but may be an interesting academic endeavour.] .

5. Conclusion

[Initially the impact of network width and depth was investigated to see how these variables effected generalization. It was found that increasing the width past a certain point for a 1 layer network left it prone to overfitting and the model generalized poorly. It was discussed that this is indicated by a growing validation train error gap and a drop in validation accuracy. Subsequently, model depth was investigated and similar conclusions were drawn. Namely that deeper networks generalized worse up to a point, however increasing the depth past 2 layers did not seem to massively increase the overfitting. It was discussed that both these phenomena were most likely due to an increase in the number of network parameters, increasing its capacity and leaving it prone to overfitting. These findings were consistent with the subject literature and is a well known phenomena in machine learning (Goodfellow et al., 2016, Ch. 5.2) (Bejani & Ghatee, 2021) (Zhang et al., 2016).

Next, a range of regularization techniques were investigated and compared against a baseline model. It was found that employing Dropout layers with a hyperparameter of $\lambda_d = 0.97$ and a Label Smoothing with $\alpha = 0.1$ both gave the best validation accuracies of 85.4% (baseline was 83.7%). L2 regularization was also found to yield a better validation (85.1 %) accuracy than baseline model. The best performing model was determined to be the Dropout model. It was chosen over the Label Smoothing as it had lower train and validation errors. This yielded a test accuracy of 84.4%, indicating that the technique allowed the previously overparameterised network to generalise well.

Finally, further investigations were proposed. It was decided that further efforts should go towards investigating if combinations of the regularizing techniques lead to better generalization, or if their effects oppose one another. It was also discussed that finding the optimal hyperparameter values for the successful techniques would be beneficial. There would however need to be techniques implemented (such as K-Fold cross validation) to ensure one wasn't fitting hyperparameters to a spe-

cific validation set. It was also discussed that training well past the point of overfitting may also yield spontaneous generalization (Power et al., 2022).] .

References

- Bejani, Mohammad Mahdi and Ghatee, Mehdi. A systematic review on overfitting control in shallow and deep neural networks. *Artificial Intelligence Review*, 54:6391–6438, 12 2021. ISSN 15737462. doi: 10.1007/s10462-021-09975-1.
- Goodfellow, Ian, Bengio, Yoshua, and Courville, Aaron. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- Guo, Chuan, Pleiss, Geoff, Sun, Yu, and Weinberger, Kilian Q. On calibration of modern neural networks. *34th International Conference on Machine Learning, ICML 2017*, 3:2130–2143, 6 2017. URL <https://arxiv.org/abs/1706.04599v2>.
- He, Kaiming, Zhang, Xiangyu, Ren, Shaoqing, and Sun, Jian. Deep residual learning for image recognition. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2016-December:770–778, 12 2015. ISSN 10636919. doi: 10.1109/CVPR.2016.90. URL <https://arxiv.org/abs/1512.03385v1>.
- Lee, Jaehoon, Xiao, Lechao, Schoenholz, Samuel S., Bahri, Yasaman, Novak, Roman, Sohl-Dickstein, Jascha, and Pennington, Jeffrey. Wide neural networks of any depth evolve as linear models under gradient descent. *Journal of Statistical Mechanics: Theory and Experiment*, 2020, 2 2019. doi: 10.1088/1742-5468/abc62b. URL <http://arxiv.org/abs/1902.06720><http://dx.doi.org/10.1088/1742-5468/abc62b>.
- Matejka, Pavel, Zhang, Le, Ng, Tim, Mallidi, Sri Harish, Glembek, Ondrej, Ma, Jeff, and Zhang, Bing. Neural network bottleneck features for language identification. In *Odyssey 2014: Speaker and Language Recognition Workshop*, pp. 299–304. International Speech Communication Association, 2014. doi: 10.21437/odyssey.2014-45.
- Ng, Andrew Y. Feature selection, l_1 vs. l_2 regularization, and rotational invariance. In *Proceedings of the twenty-first international conference on Machine learning*, pp. 78, 2004.
- Power, Alethea, Burda, Yuri, Edwards, Harri, Babuschkin, Igor, and Misra, Vedant. Grokking: Generalization beyond overfitting on small algorithmic datasets. 1 2022. URL <http://arxiv.org/abs/2201.02177>. This paper reproduces the process of Grokking and that this process can happen well past the point of overfitting. It also studies generalization as a function of dataset size and finds that increasing amounts of optimization is required for generalization on smaller datasets. It is relevant for my IRR as it describes, recreates and investigates the conditions under which Grokking occurs.
- Srivastava, Nitish, Hinton, Geoffrey, Krizhevsky, Alex, Sutskever, Ilya, and Salakhutdinov, Ruslan. Dropout:

a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1): 1929–1958, 2014.

Zhang, Chiyuan, Bengio, Samy, Hardt, Moritz, Recht, Benjamin, and Vinyals, Oriol. Understanding deep learning requires rethinking generalization. 11 2016. URL <http://arxiv.org/abs/1611.03530>.